

- Подход **Mobile First**: базовые стили (без медиазапроса) пишутся для мобильного экрана, затем добавляются `@media (min-width: 768px) { ... }` и `@media (min-width: 1200px) { ... }`, расширяющие/переопределяющие стили. Медиазапросы вида `@media (max-width: ...)`, написанные «от десктопа к мобильному», не принимаются.
- Обязательно использование и Grid, и Flexbox в одном проекте (Grid — для крупной сетки страницы, Flexbox — для компонентов). Проект, целиком сверстанный только на Flexbox без единого `display: grid`, не засчитывается.
- Файлы: `task2/index.html`, `task2/style.css`. Изображения — либо ссылками, либо в подпапке `task2/img/`.
- Сходство с макетом Figma должно быть в разумных пределах: отступы, шрифты, цвета — близко к оригиналу (пиксель-в-пиксель не требуется, но пропорции и структура должны совпадать).
- Проверить вёрстку на реальном телефоне или через режим устройства в DevTools (Ctrl+Shift+M) — не должно быть горизонтального скролла, элементы не должны вылезать за экран, размеры кнопок/полей — не менее 44×44px (touch-friendly).

Что сдавать: ссылку на макет Figma, использованный вами (если преподаватель выдавал несколько на выбор — указать, какой), и папку `task2/`.

Критерии проверки: есть все три контрольные точки, стили написаны mobile-first, используются и Grid, и Flexbox по назначению, форма и меню оформлены, нет горизонтального скролла на мобильной ширине.

Лабораторная работа №3. Основы JS и DOM: Todo-list

Цель: научиться работать с DOM-деревом, обрабатывать события и управлять состоянием интерфейса на чистом JavaScript.

Что нужно сделать: Реализовать интерактивный список задач (без сохранения данных между перезагрузками страницы — localStorage здесь использовать не нужно, это будет в лабораторной №9).

Функциональность:

1. Поле ввода (`<input type="text">`) и кнопка «Добавить» (или отправка по Enter — сделать оба варианта).
2. При добавлении задачи (текст не должен быть пустым — если поле пустое, задача не добавляется и/или показывается предупреждение) она появляется в списке под полем ввода.
3. У каждой задачи в списке должно быть:
 - чекбокс или кнопка «Выполнено» — по клику задача визуально помечается как выполненная (например, зачёркивание текста через CSS-класс `.completed { text-decoration: line-through; opacity: 0.6; }`);
 - кнопка «Удалить» — удаляет задачу из списка.

4. Счётчик над или под списком: «Осталось: N, Выполнено: M».
5. Фильтр из трёх кнопок/вкладок: «Все», «Активные», «Завершённые». При клике на фильтр список визуально показывает только соответствующие задачи (остальные скрываются, например через `display: none`, а не удаляются из данных).

Технические требования (обязательные):

- Вся логика — в одном файле `script.js`, подключённом к `task3/index.html` через `<script src="script.js" defer></script>`.
- HTML-разметка отдельных задач **генерируется в JavaScript** (через `document.createElement` или сборку строки и `innerHTML`), не прописывается заранее вручную в HTML.
- Список задач должен храниться в JS в виде массива объектов, например:

```
// { id: 1, text: "Сделать лабу", completed: false }
```

Отрисовка списка на экране должна происходить на основе этого массива (функция `render()`, которая перерисовывает список при любом изменении данных).

- Использовать методы массивов `map` / `filter` / `forEach` там, где это уместно (например, `filter` — для отображения по текущему фильтру, `map` — при рендере).
- Каждой задаче должен присваиваться уникальный идентификатор (можно просто счётчик, увеличивающийся на 1, или `Date.now()`).
- Обработка событий — через `addEventListener` (не через атрибуты `onclick="..."` в HTML).
- В консоли браузера (F12) не должно быть ни одной ошибки при использовании приложения (добавление, удаление, отметка выполнения, переключение фильтров).

Что сдавать: `task3/index.html`, `task3/style.css`, `task3/script.js`.

Критерии проверки: все 5 пунктов функциональности работают, данные хранятся в массиве и рендерятся из него, нет `onclick` в HTML, нет ошибок в консоли.

Лабораторная работа №4. Игра «Быки и коровы»

Цель: закрепить работу с состоянием приложения, массивами/строками и условной логикой без использования Canvas.

Правила игры (если незнакомы): компьютер загадывает число из 4 неповторяющихся цифр (от 0 до 9). Игрок вводит своё предположение — тоже 4 цифры. После каждой попытки показывается результат: количество «быков» (цифра угадана и стоит на своём месте) и «коров» (цифра есть в загаданном числе, но стоит на другом месте).

Что нужно сделать:

1. При загрузке страницы (или по кнопке «Новая игра») генерируется случайное 4-значное число с неповторяющимися цифрами.
2. Поле ввода для попытки игрока + кнопка «Проверить».